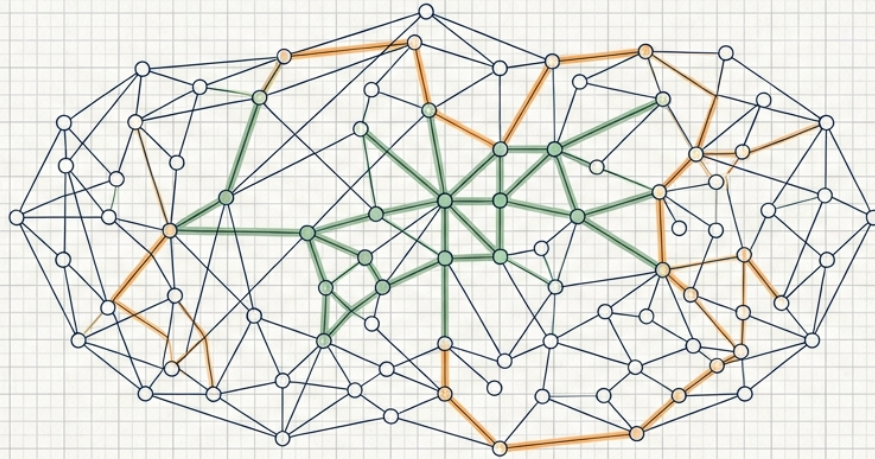


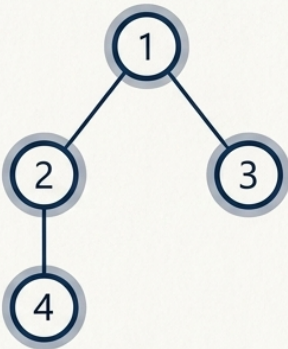
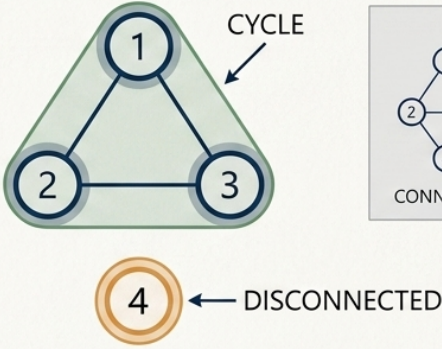
The Cartographer's Blueprint

Mastering Graph Algorithms for the Technical Interview

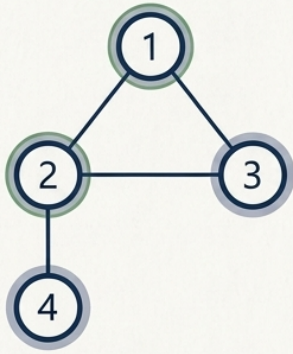


A visual playbook from unweighted traversals to Minimum Spanning Trees

THE PARADIGM SHIFT: BREAKING THE HIERARCHY

THE TREE		THE GRAPH	
 TREE		 GRAPH	
STRUCTURE	Connected, acyclic hierarchy	Unrestricted network.	
EDGES	Strictly $n - 1$ edges	Ranging from 1 to $O(V ^2)$ edges.	
PATHS	Exactly 1 path between any two nodes	Multiple overlapping paths or no paths at all.	
A TREE IS SIMPLY A STRICT, SPECIALIZED SUBSET OF A GRAPH. TO MASTER GRAPHS, WE MUST LEARN TO MAP ENVIRONMENTS THAT ALLOW CYCLES AND ISLANDS.			

The Architect's Schema: Adjacency Lists



```
{  
  "1": [2, 3],  
  "2": [1, 3, 4],  
  "3": [1, 2],  
  "4": [2]  
}
```

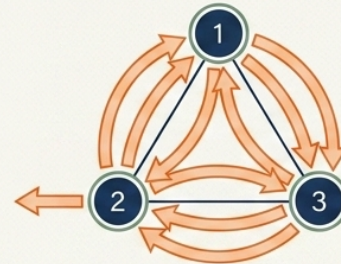
Graphs are rarely given as concrete objects. They are maps of relationships. For every vertex, we store an array of its neighbors.

Blueprint Note:

In interviews, even if a graph is implicit, all traversal algorithms ultimately require a single functional concept:
concept: `get_neighbors(node)`.

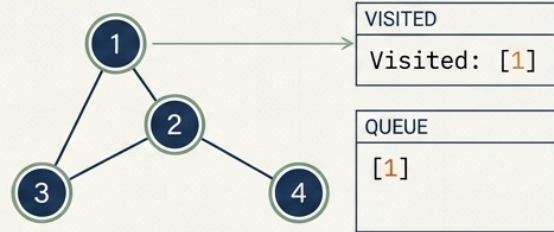
The Threat of Infinity

THE THREAT



Because graphs contain cycles, traditional tree traversal guarantees an infinite loop. We solve this by dropping breadcrumbs.

THE SOLUTION



The Blueprint: The visited variable. This can be a Hash Set or a boolean array. It answers existence queries in $O(1)$ constant time, ensuring we never process the same node twice.

Code Diff: Tree BFS vs. Graph BFS

Tree BFS

```
public void bfs(Node root) {
    ArrayDeque<Node> queue = new ArrayDeque<>();
    queue.add(root);
    while (queue.size() > 0) {
        Node node = queue.pop();
        for (Node child : node.children) {
            queue.add(child);
        }
    }
}
```

Graph BFS

```
public void bfs(Node root) {
    ArrayDeque<Node> queue = new ArrayDeque<>();
    queue.add(root);
    Set<Node> visited = new HashSet<>();
    visited.add(root);
    while (queue.size() > 0) {
        Node node = queue.pop();
        for (Node neighbor : getNeighbors(node)) {
            if (visited.contains(neighbor)) { continue; }
            queue.add(neighbor);
            visited.add(neighbor);
        }
    }
}
```

The algorithm is identical. The only architectural difference is the memory required to track where we have been.

Unweighted Exploration: BFS vs. DFS

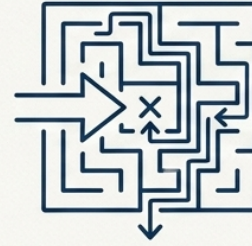


BFS

Mechanism: Level-by-level (Queue).

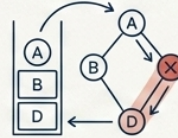
Superpower: Finding the absolute shortest path in unweighted networks.

Ideal Target: Graphs of unknown or infinite size (e.g., Knight's shortest path).



DFS

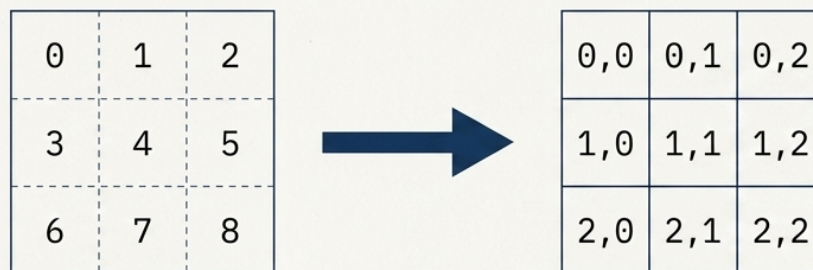
Mechanism: Plunge to the deepest node, then backtrack (Recursion/Stack).



Superpower: Memory efficiency on wide graphs (no massive queues).

Ideal Target: Searching for distant exits or exhaustively mapping connected components.

The Grid Reality: Implicit Graphs

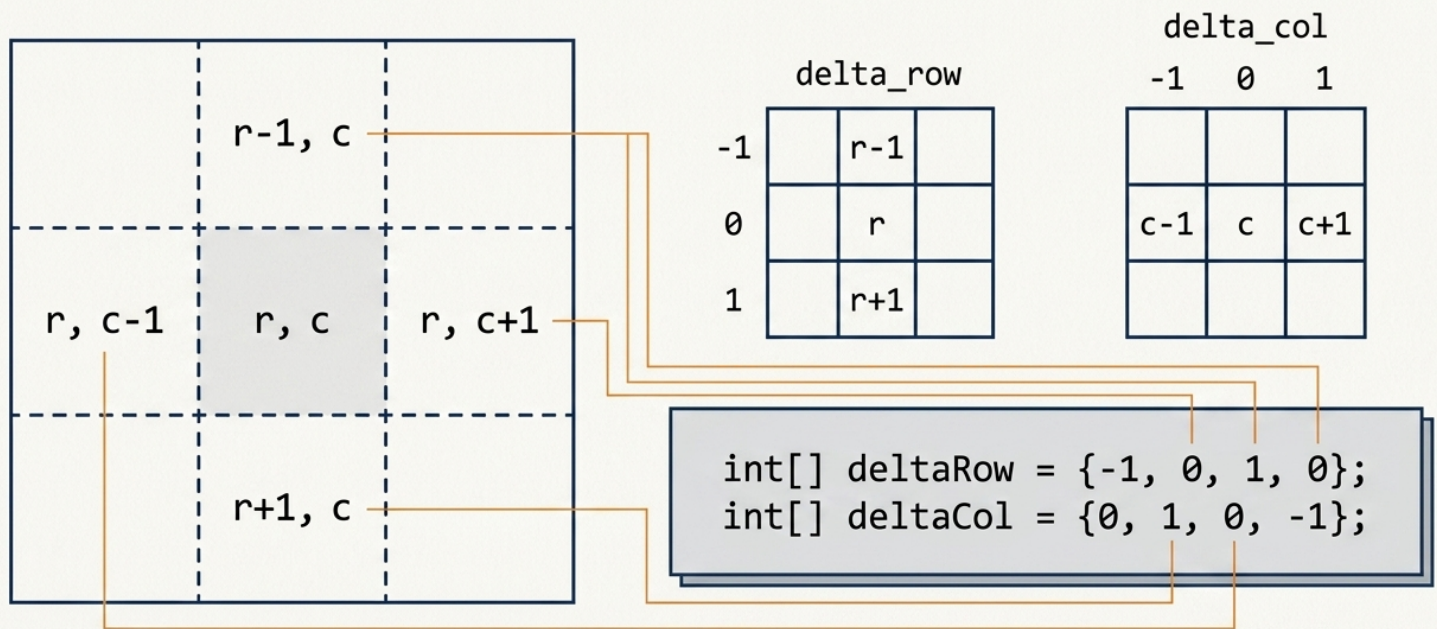


Core Concept: In matrix problems (like 'Number of Islands'), nodes are (row, col) coordinates. The edges are implicit boundaries between adjacent cells.

The Catch: We don't build an adjacency list upfront. We compute neighbors dynamically on the fly.

State Tracking: Instead of a complex Hash Set, visited states can often be tracked by mutating the input grid directly (e.g., flipping a 1 to a 0).

Cardinal Navigation on the Grid



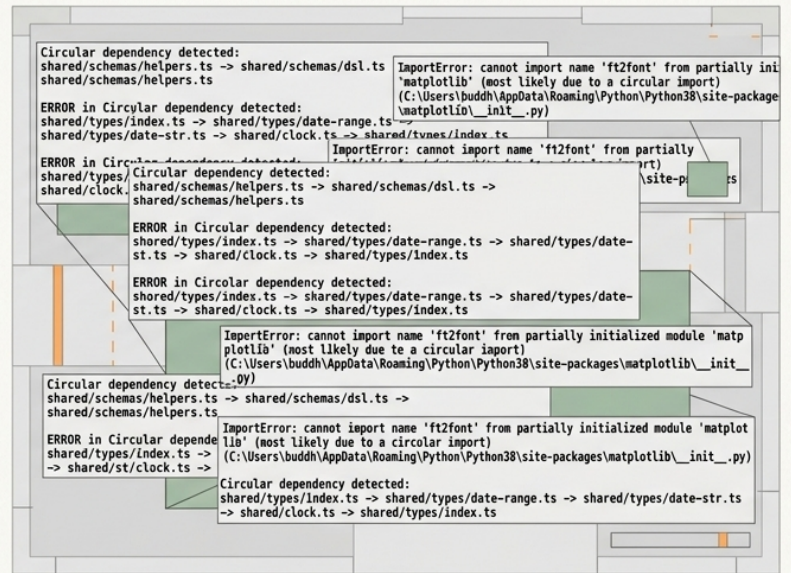
Takeaway: Iterate i from 0 to 3. Add $\text{deltaRow}[i]$ and $\text{deltaCol}[i]$ to current coordinates. Filter out any results that fall outside grid boundaries ($0 \leq r < \text{numRows}$). This perfectly replicates `get_neighbors()` for grids.

Directed Dependencies & The Deadlock

Explanation: When edges have a one-way direction (arcs), they represent dependencies. Node V depends on Node W.

The Rule of Acyclicity: A graph with directed cycles has no logical ordering. If A requires B, B requires C, and C requires A, the system halts.

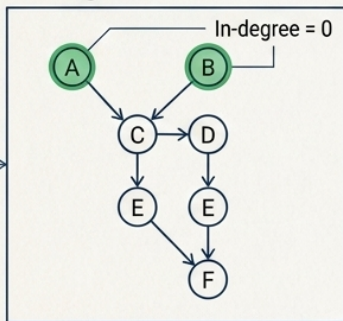
The Mission: We need an algorithm to determine a valid sequence of execution where every prerequisite is met before a node is processed.



Kahn's Algorithm: Peeling the Onion

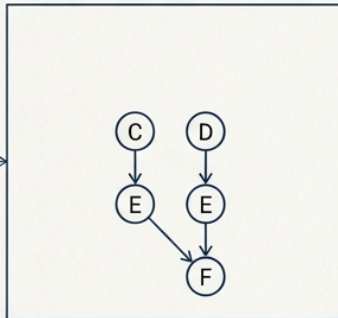
Analogy: Putting on clothes. You cannot put on shoes (in-degree = 1) until the socks (in-degree = 0) are processed.

Panel 1: Initial DAG & In-degree 0



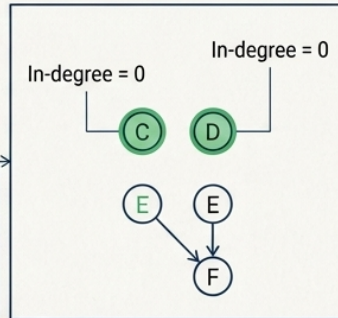
Start with nodes having no incoming edges.

Panel 2: Remove Nodes & Outgoing Edges



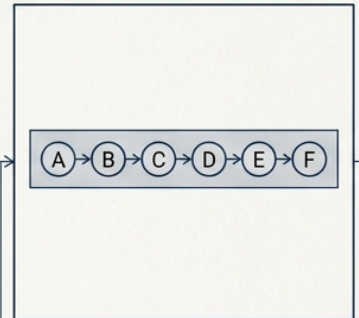
Remove the green nodes and their outgoing edges.

Panel 3: Highlight Newly Freed Nodes



Identify new nodes with in-degree 0.

Panel 4: The Topological Sort

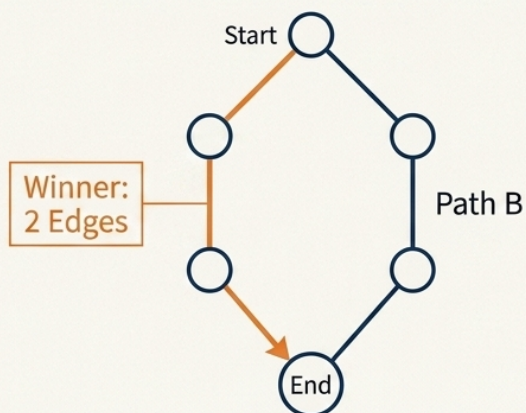


Resulting linear ordering of nodes.

The Engine: We process nodes exactly like BFS, but instead of pushing all neighbors into the queue, we only push a neighbor when its in_degree hits exactly 0.

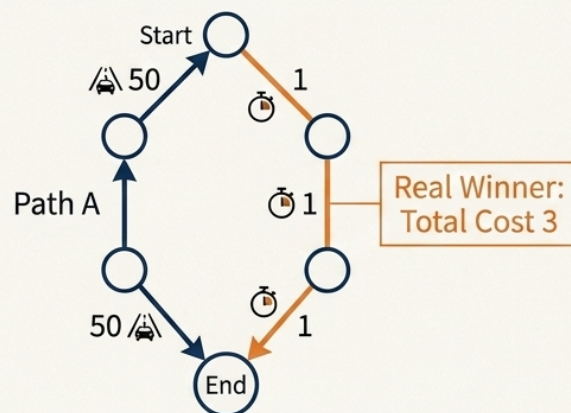
The Weight of the World

UNWEIGHTED GRAPH (BFS View)



BFS finds the fewest edges, regardless of cost.

WEIGHTED GRAPH (GPS View)



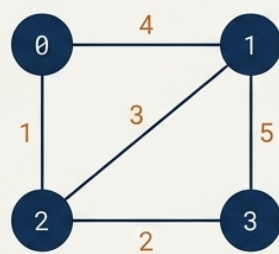
In reality, some paths are vastly more expensive.

The Flaw in BFS: BFS assumes every edge costs exactly 1 unit of time/distance. In the real world, connections have unequal friction.

The Paradigm Shift: The first time we reach a node in a weighted graph does not guarantee we found the minimum distance. We may find a shorter detour later.

The Solution: We must abandon the FIFO Queue and adopt a Priority Queue to constantly re-evaluate the cheapest path.

Dijkstra's Engine: The Priority Queue

Weighted Graph	Priority Queue	Distances																				
	<table><tr><td>Priority</td><td>INF</td><td>INF</td><td>INF</td><td>0</td></tr><tr><td>Node</td><td>3</td><td>2</td><td>1</td><td>0</td></tr></table>	Priority	INF	INF	INF	0	Node	3	2	1	0	<table><tr><td>Node</td><td>0</td><td>1</td><td>2</td><td>3</td></tr><tr><td>Distance</td><td>0</td><td>INF</td><td>INF</td><td>INF</td></tr></table>	Node	0	1	2	3	Distance	0	INF	INF	INF
Priority	INF	INF	INF	0																		
Node	3	2	1	0																		
Node	0	1	2	3																		
Distance	0	INF	INF	INF																		

Step 1 (Initialization)	Step 2 (Extract Min)	Step 3 (Relaxation)
Source node distance is 0. All other nodes are set to INFINITY. dist[0]=0, dist[v]=INF for v≠0.	Always pop the unvisited node with the lowest current distance. min_node = PQ.pop().	For all neighbors v of current node u, calculate alt = dist[u] + weight(u,v). If alt < dist[v], update dist[v] and push to the priority queue.

Time Complexity:
 $O((V+E) * \log(V))$ due to logarithmic heap operations.

The Dijkstra Blueprint (With Optimizations)

```
while (pq.size() > 0) {  
    Entry t = pq.poll();  
    int dist = t.getKey();  
    int node = t.getValue();
```

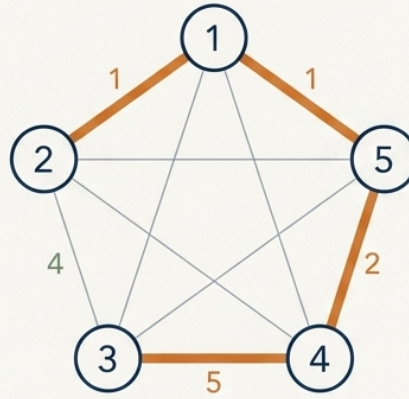
CRITICAL: Skip stale entries to simulate $O(1)$ decrease-key.

```
    if (dist > distances.get(node)) continue;
```

```
    for (Entry neighbor : getNeighbors(graph, node)) {  
        // ... calculate and push shorter paths ...  
    }  
}
```

Implementation Note: Instead of searching a heap to update a value (**decrease_priority**), simply add the newly found shorter distance to the heap. The stale, longer distance will naturally be ignored when popped later.

The Blueprint of Optimal Connections: MST



The Mission

Given a **weighted network**, select edges to connect **every single node** while keeping the total weight as low as possible.

The Rules

- 1) Must include **all V** nodes.
- 2) Must use exactly **$V - 1$** edges.
- 3) Must contain **zero cycles**.

The Analogy

Laying down the minimum amount of wildly expensive **fiber-optic cable** needed to ensure every city is on the grid.

The Two Architects: Kruskal vs. Prim

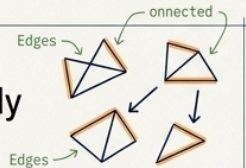
Kruskal's Algorithm

Philosophy

Edge-centric. Sort all edges globally.

Growth Pattern

Forms multiple disjoint components that eventually merge into one.



Mechanism

Uses Disjoint-Set (Union-Find) to prevent cycles.

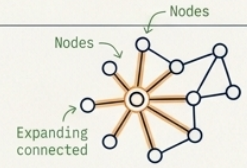
Prim's Algorithm

Philosophy

Node-centric. Start anywhere.

Growth Pattern

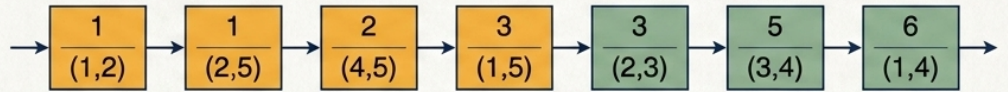
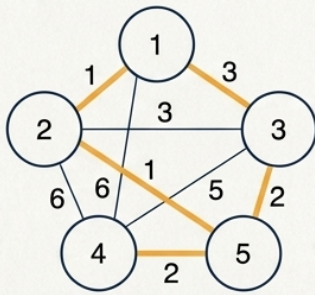
Grows outward from a single component like a mold.



Mechanism

Uses a Priority Queue to constantly pick the cheapest edge connected to the existing tree.

Kruskal's Execution & Union-Find



Step 1

Sort all edges by weight globally $O(E \log E)$.

The entire collection of potential connections is ordered from lowest cost to highest cost.

Step 2

Iterate through the sorted list.

For each edge (u, v) , ask the Disjoint-Set:
"Do these two nodes already belong to the same group?"

```
if (dsu.find(u) != dsu.find(v))
```

Step 3

If $dsu.find(A) == dsu.find(B)$, skip it (adding it creates a cycle).

If not, $dsu.union(A, B)$ and add the edge to the tree. This connects two previously disjoint components.

```
dsu.union(u, v);  
mst_edges.add((u, v));
```

Optimization: Path compression and union-by-rank make disjoint-set operations run in near-constant amortized time, ensuring efficiency.

The Architect's Synthesis: Graph Interview Cheat Sheet

Goal	Tool	Complexity
Shortest path in unweighted graph/grid	BFS	$O(V+E)$
Connected components / Deep exhaustive search	DFS	$O(V+E)$
Handle dependency chains / ordering	Kahn's (Topological Sort)	$O(V+E)$
Shortest path in weighted graph	Dijkstra's (Priority Queue)	$O((V+E) \log V)$
Connect all nodes with minimum total weight	Kruskal's / Prim's	$O(E \log E)$

Master these five templates, and the graph interview is solved.